Internal Code Review Benchmark System (GitLab + LLMs)

Overview

This document summarizes a complete design for building an internal **code review benchmarking system** inspired by the Martian Code Review Benchmark, adapted for private GitLab/GitHub repositories.

It covers both:

- Offline benchmarking (historical PR analysis)
- Online benchmarking (live PR monitoring)

It also defines:

- Review agents (GPT-5, Claude, Hybrid)
- Ground truth construction
- Scoring methodology
- GitLab collector implementation
- Workflow design
- Optional PR automation strategy

---

1. Core Concept

The system evaluates AI code reviewers by comparing their findings against real engineering outcomes.

Key idea

A PR becomes a dataset entry:

- Input: code diff at review time
- Output: AI-generated findings
- Label: human outcome (fixes, comments, incidents)

---

2. Review Agents

A "review agent" is any system that inspects a PR and produces findings.

2.1 LLM-based agents

GPT-5 Reviewer

- Single-pass PR review
- Focus: correctness, security, performance

Claude Reviewer

- Same role, different model

Gemini Reviewer (optional extension)

2.2 Hybrid Agent (Recommended Strongest)

Pipeline:

1. Run static analyzers
2. Semgrep
3. CodeQL
4. Feed results into LLM
5. LLM deduplicates + adds reasoning

Example:

```
PR
 ├─ Semgrep
 ├─ CodeQL
 └─ GPT-5 synthesis
```

2.3 Static Analysis Agents

- Semgrep
- CodeQL
- SonarQube (optional)

Used as baseline or hybrid inputs.

---

3. Offline Benchmark (Historical PRs)

3.1 Goal

Evaluate AI reviewers using past PR outcomes.

3.2 Dataset Selection

Include:

- Bug fixes
- Security fixes
- Feature changes
- Refactors

Exclude:

- Dependency bumps
- Formatting-only PRs
- Docs-only changes

Recommended:

- MVP: 100–200 PRs
- Production: 500–2000+ PRs

3.3 PR Reconstruction

For each PR store:

```
{
  "repo": "service-a",
  "pr_id": 123,
  "diff": "...",
  "files": [...],
  "commits": [...],
  "discussions": [...]
}
```

Critical: reconstruct the **state at review time**, not current state.

3.4 Ground Truth Construction

Use multiple signals:

1. Review comments

- "Possible null pointer"

2. Requested changes

- "Changes requested"

3. Follow-up commits

- "Fix race condition"

4. Hotfixes

- Production fixes linked to PR

5. Incidents

- Incident tracking systems

3.5 Matching Findings

Use LLM-based matching:

```
Agent Finding:
...

Ground Truth:
...

Are they the same issue?
YES/NO
Confidence: 0–100
```

3.6 Metrics

Precision

```
valid findings / total findings
```

Recall

```
matched ground truth / total ground truth
```

Severity Weighted Recall

Weights:

- Critical = 10
- High = 5
- Medium = 2
- Low = 1

---

4. Online Benchmark

4.1 Flow

```
PR Opened
   ↓
Snapshot PR
   ↓
Run Review Agents
   ↓
Store Findings
   ↓
Wait 7–30 days
   ↓
Evaluate outcomes
```

4.2 Key Design Rule

Do NOT expose AI findings to developers during benchmark phase.

Avoid contamination of ground truth.

---

5. GitLab Collector (Python Implementation)

5.1 Purpose

Extract Merge Requests and related data from GitLab API.

5.2 Implementation

```python
from __future__ import annotations

import json
import os
from pathlib import Path
from typing import Any
import requests


class GitLabCollector:
    """
    Collects GitLab Merge Requests for benchmarking.
    """

    def __init__(
        self,
        base_url: str,
        private_token: str,
        output_dir: str = "dataset",
    ):
        self.base_url = base_url.rstrip("/")
        self.output_dir = Path(output_dir)

        self.session = requests.Session()
        self.session.headers.update(
            {"PRIVATE-TOKEN": private_token}
        )

    def _get(self, endpoint: str, params: dict | None = None) -> Any:
        url = f"{self.base_url}/api/v4{endpoint}"
        response = self.session.get(url, params=params, timeout=60)
        response.raise_for_status()
        return response.json()
```

```
    def get_merge_request(self, project_id: int, mr_iid: int) -> dict:
        return self._get(f"/projects/{project_id}/merge_requests/{mr_iid}")

    def get_changes(self, project_id: int, mr_iid: int) -> dict:
        return self._get(f"/projects/{project_id}/merge_requests/{mr_iid}/
changes")

    def get_discussions(self, project_id: int, mr_iid: int) -> list:
        return self._get(f"/projects/{project_id}/merge_requests/{mr_iid}/
discussions")

    def get_commits(self, project_id: int, mr_iid: int) -> list:
        return self._get(f"/projects/{project_id}/merge_requests/{mr_iid}/
commits")

    def collect_merge_request(self, project_id: int, mr_iid: int) -> dict:
        mr = self.get_merge_request(project_id, mr_iid)
        changes = self.get_changes(project_id, mr_iid)
        discussions = self.get_discussions(project_id, mr_iid)
        commits = self.get_commits(project_id, mr_iid)

        return {
            "project_id": project_id,
            "mr_iid": mr_iid,
            "metadata": mr,
            "changes": changes,
            "discussions": discussions,
            "commits": commits,
        }

    def save_snapshot(self, snapshot: dict) -> Path:
        project_id = snapshot["project_id"]
        mr_iid = snapshot["mr_iid"]

        target_dir = self.output_dir / str(project_id) / str(mr_iid)
        target_dir.mkdir(parents=True, exist_ok=True)

        file_path = target_dir / "snapshot.json"

        with open(file_path, "w", encoding="utf-8") as f:
            json.dump(snapshot, f, indent=2, ensure_ascii=False)

        return file_path
```

5.3 Usage

```
collector = GitLabCollector(
    base_url="https://gitlab.company.com",
    private_token=os.environ["GITLAB_TOKEN"],
)

snapshot = collector.collect_merge_request(123, 456)
collector.save_snapshot(snapshot)
```

---

6. Full Offline Benchmark Pipeline

Steps

1. Collect historical PRs
2. Normalize snapshots
3. Extract ground truth
4. Run review agents:
5. GPT-5
6. Claude
7. Hybrid (Semgrep + CodeQL + LLM)
8. Match findings
9. Score precision/recall
10. Build leaderboard

---

7. Hybrid Reviewer Architecture

```
PR
 ├─ Semgrep
 ├─ CodeQL
 └─ GPT-5
        ↓
   Deduplicate findings
   Add reasoning
   Assign severity
```

---

8. PR Automation Strategy (IMPORTANT)

Do NOT auto-create PRs during benchmarking

Risk:

- Contaminates evaluation
- Biases ground truth

Safe workflow

Stage 1

Silent benchmarking only

Stage 2

Findings → human review queue

Stage 3

After validation:

- Create GitLab issue OR
- Create PR (optional)

Optional Advanced Workflow

```
Finding
  ↓
30-day no-fix check
  ↓
LLM revalidation
  ↓
Engineer approval
  ↓
Issue or PR creation
```

---

9. Final System Vision

The mature system becomes:

- Benchmark platform
- Code review intelligence layer
- Production defect predictor
- Optional auto-fix engine

Summary

This system enables:

- Comparing GPT-5 vs Claude vs Hybrid reviewers
- Measuring real defect detection capability
- Building trusted internal benchmarks
- Eventually enabling safe automation of PR creation

End of document.